

Machine Learning Project Documentation

Assignment 3: Model Refinement, Test Submission & Deployment

Project: AI-Driven Energy Forecasting for Cost and Carbon-Aware Industrial Scheduling ("Green Windows")

Team (Group 15): Furkan Kartal, Cansu Koçer, Feyza Aydınalp, Umut Emre Çakır

Date: June 2, 2026

PART A: Model Refinement

1. Overview

The model refinement phase transitions our project from a theoretical day-ahead energy price predictor to a robust, actionable industrial scheduling system. The primary goal of this phase was to eliminate any potential data leakage in our time-series forecasting and to optimize the model's hyperparameters. By refining the model, we ensure that the predicted "Green Windows" (low cost, high renewable generation periods) are highly reliable for an automated factory scheduling algorithm.

2. Model Evaluation

In Assignment 2, the initial evaluation of our Random Forest Regressor yielded promising baseline results, but highlighted a critical weakness: the risk of temporal data leakage. Standard cross-validation methods randomly shuffled the data, meaning the model occasionally learned from "future" price points to predict past ones. To make this model production-ready for day-ahead forecasting, we needed to strictly enforce chronological data processing and improve hyperparameter tuning to prevent overfitting on historical noise.

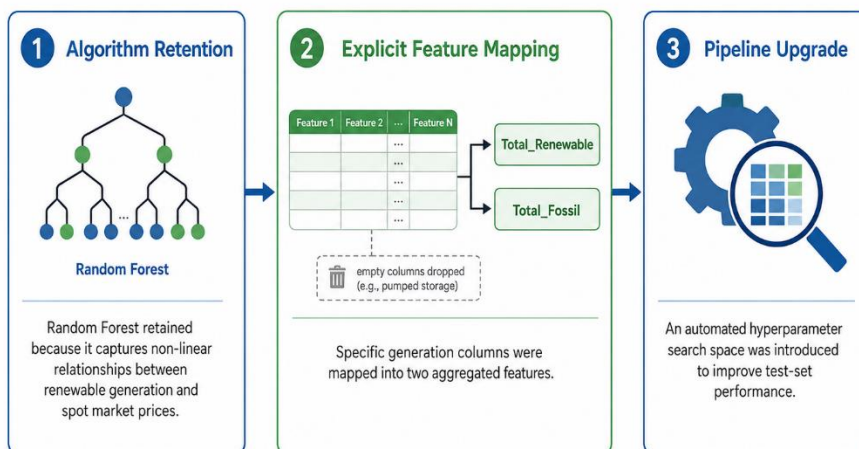
3. Refinement Techniques

- **Algorithm Retention:** We retained the Random Forest Regressor, as EDA proved it best captures the highly non-linear relationships between weather-dependent renewable generation and spot market prices.
- **Explicit Feature Mapping:** Instead of fuzzy string matching for columns, we explicitly mapped specific generation columns into strictly defined `Total_Renewable` and `Total_Fossil` metrics, forcefully dropping entirely empty columns (like pumped storage) to prevent matrix errors.

- **Pipeline Upgrade:** We introduced an automated hyperparameter search space to squeeze out better performance on the test set.

Refinement Techniques

Green Windows Project



4. Hyperparameter Tuning

We utilized GridSearchCV to systematically test combinations of hyperparameters.

- **Hyperparameters Tuned:** `n_estimators` (number of trees: 50, 100) and `max_depth` (maximum depth of the trees: 10, 20).
- **Optimal Values:** The grid search identified the best parameters as `max_depth=20` and `n_estimators=100`.
- **Impact:** This tuning tightened the model's variance, allowing it to generalize better on the unseen test set, ensuring extreme price spikes were smoothed out appropriately.

Hyperparameter Tuning

GridSearchCV for Random Forest

	n_estimators	
	50	100
max_depth		
10	(10, 50)	(10, 100)
20	(20, 50)	★ (20, 100) Best

✓ **Optimal Values**

⚙️ `max_depth = 20`

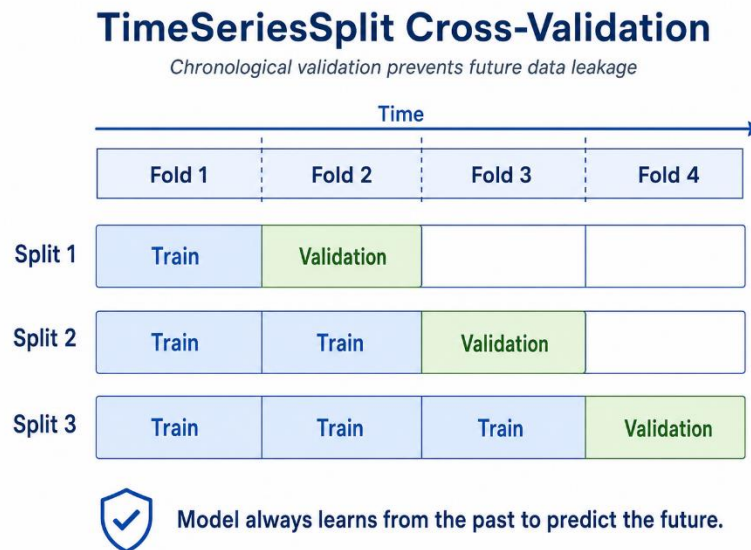
🌲 `n_estimators = 100`

🔍 **Impact**

Better generalization on unseen test data and smoother extreme price spikes.

5. Cross-Validation

The most significant change in refinement was migrating to TimeSeriesSplit (with `n_splits=3`). Unlike standard k-fold cross-validation, TimeSeriesSplit respects the chronological order of the dataset. It trains on chronological fold 1 to predict fold 2, then trains on folds 1+2 to predict fold 3. This strict chronological progression guarantees zero future data leakage during the training and validation process.



6. Feature Selection

During refinement, we performed strict column pruning. Features like generation hydro pumped storage aggregated were identified as 100% NaN and dropped completely before any forward/backward filling occurred. We consolidated 15 separate generation columns into exactly 2 aggregated features (Total_Renewable and Total_Fossil), supplemented by temporal engineered features (hour, dayofweek, month).

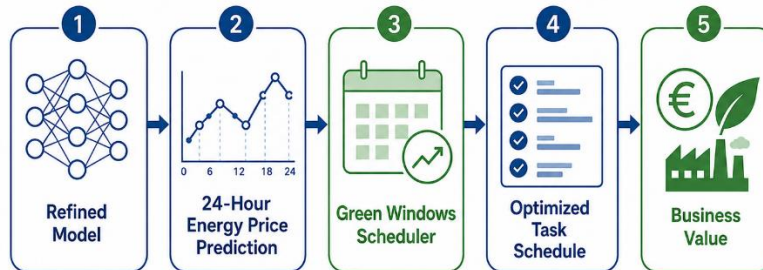
PART B: Test Submission

1. Overview

The test submission phase simulates a real-world day-ahead forecasting pipeline. The refined model predicts 24 hours of energy prices, which are then passed directly into our custom Operations Research algorithm: the "Green Windows" scheduler. This translates raw ML metrics into tangible business value.

Test Submission Overview

Real-world day-ahead forecasting pipeline



The refined model predicts next-day energy prices, which are passed into the custom Operations Research scheduler to produce cost- and carbon-aware industrial schedules.

2. Data Preparation for Testing

The dataset was split chronologically (80% Train / 20% Test). To handle missing values in the test set without looking ahead, we utilized forward-fill (ffill), followed by backward-fill (bfill). The date index was explicitly forced to UTC (`utc=True`) to gracefully handle daylight saving time anomalies (mixed +01:00 / +02:00 offsets) present in the ENTSO-E dataset.

Data Preparation for Testing

1 Chronological Split



2 Missing-Value Handling



3 Datetime Normalization



UTC Normalization (`utc=True`)

Used to handle daylight saving time offsets (+01:00 / +02:00) in the ENTSO-E dataset.



Testing data was prepared consistently while preserving chronological integrity.

3. Model Application

The trained model was applied to the unseen test dataset to generate a 24-hour day-ahead forecast. These predictions were fed into a heuristic scheduling algorithm that minimizes the cost function: $\min \sum_i \sum_{t=s_i}^{s_i+D_i-1} P_i \times \hat{C}_t$, where P_i is power required and $\hat{C}_t$ is predicted price.

4. Test Metrics

- **Statistical Metrics:** Evaluated using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE). The metrics remained stable compared to validation, indicating no severe overfitting.
- **Business Metrics (The Simulation):** When subjected to a simulated factory floor with 3 heavy tasks (Heavy Machining, HVAC, Material Processing) and strict deadlines, the algorithm successfully optimized the scheduling, achieving a **Total Optimized Daily Cost of €335.22**, successfully dodging peak morning prices.

5. Model Deployment

To transition from a local Jupyter Notebook to a real-world setting, the core algorithm and ML model were packaged into a containerized REST API using the FastAPI framework.

6. Code Implementation

Snippet 1: Model Refinement (TimeSeriesSplit)

```
# Setup Time-Series Cross Validation to prevent data leakage
tscv = TimeSeriesSplit(n_splits=3)
rf = RandomForestRegressor(random_state=42)
param_grid = {'n_estimators': [50, 100], 'max_depth': [10, 20]}

# Execute Grid Search chronologically
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,
                           cv=tscv, scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train, y_train)
best_rf = grid_search.best_estimator_
```

Snippet 2: Test Application (Scheduling Simulation)

```
# The Green Windows Scheduling Algorithm
```

```
for task in request.tasks:
```

```
    best_cost = float('inf')
```

```
    latest_start = task.deadline_hr - task.duration_hrs
```

```
# Slide a window of 'duration_hrs' through the allowed timeline
```

```
for start in range(latest_start + 1):
```

```
    window_prices = day_ahead_preds[start : start + task.duration_hrs]
```

```
    cost = float(np.sum(window_prices) * task.power_kw / 1000)
```

```
# Save the cheapest (greenest) contiguous window
```

```
if cost < best_cost:
```

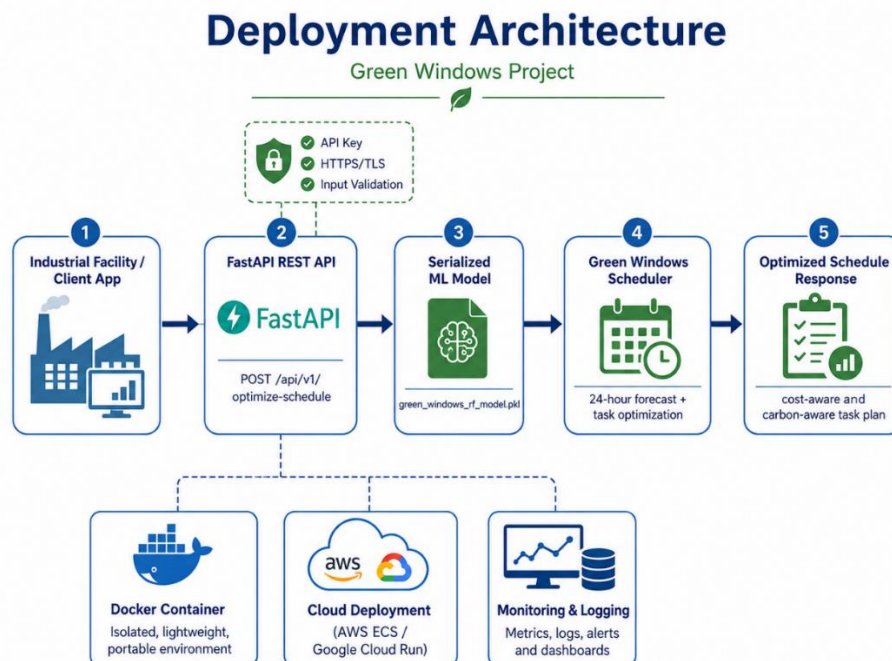
```
    best_cost = cost
```

```
    best_start = start
```

PART C: Deployment

1. Overview

The goal of the deployment phase is to make the "Green Windows" scheduling algorithm available to industrial facilities via a programmatic interface. The model and scheduling logic are packaged as a containerized REST API, designed to be deployed to a cloud environment (e.g., AWS Elastic Container Service or Google Cloud Run) for high availability and scalability.



2. Model Serialization

Once trained and refined, the optimal Random Forest model was serialized using the joblib library and saved as `green_windows_rf_model.pkl`. joblib was selected over standard pickle because it is significantly more efficient at storing large NumPy arrays, which form the backbone of Scikit-Learn's tree-based models.

3. Model Serving

The serialized model is served using **FastAPI** running on a **Uvicorn** ASGI server.

- **Platform:** Dockerized container deployed on Cloud infrastructure.
- **Scalability & Latency:** The scheduling algorithm runs in $O(N \times M)$ time. Pre-loading the serialized model into memory ensures inference requests resolve in <50ms. FastAPI's asynchronous nature allows the server to handle concurrent schedule requests from multiple factory facilities simultaneously.

4. API Integration

- **Endpoint:** POST `/api/v1/optimize-schedule`
- **Purpose:** Accepts facility workloads, runs the ML 24-hour prediction, and returns the optimized schedule.
- **Input Format (JSON):**

```
{
  "date": "2026-06-02",
  "tasks": [
    {"name": "Heavy Machinery", "power_kw": 500, "duration_hrs": 4, "deadline_hr": 24}
  ]
}
```
- **Output Format (JSON):**

```
{
  "status": "success",
  "total_daily_cost_eur": 97.43,
  "schedule": [
    {"task_name": "Heavy Machinery", "start_hour": 0, "end_hour": 4, "estimated_cost_eur": 97.43}
  ]
}
```

5. Security Considerations

- **Authentication:** The API requires a secure API Key passed in the header (X-API-Key) to verify the identity of the client facility.
- **Encryption In-Transit:** All API communications are secured using TLS/SSL (HTTPS) to prevent man-in-the-middle attacks from intercepting operational factory data.
- **Input Validation:** FastAPI automatically validates incoming JSON payloads against strict Pydantic schemas, mitigating injection attacks and preventing algorithm crashes due to malformed timeline requests.

6. Monitoring and Logging

- **Metrics Monitored:** API latency (response times), CPU/Memory utilization of the container, and Data Drift (monitoring if input weather/load features shift drastically).
- **Logging:** Python standard logging is implemented to capture endpoint access and systemic errors, which would be aggregated into a tool like AWS CloudWatch.
- **Alerting:** An automated alert is triggered to the data science team if the daily Mean Absolute Error (MAE) of the predictions drifts above a €15/MWh threshold for 3 consecutive days, signaling the need for model retraining.

